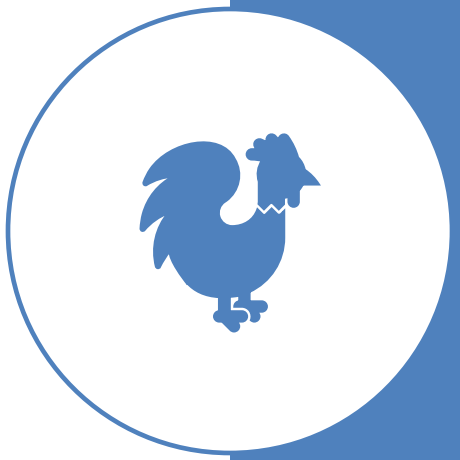
C++ Object-Oriented Programming Concepts

Er. Piyush Pant
Mahendra Ratna Campus
Date: June 2025

Definition of Class and Object

- A **class** is a blueprint or template that defines **what data** (attributes) and **what behaviors** (functions) objects of that class will have.
- It's like a plan or design.



What is an **Object**?

- An **object** is a specific instance of a class.
- It is created using the class blueprint.
- Each object has its own data but shares the structure and behavior defined by the class.

Syntax and Simple Example

```
#include <iostream>
using namespace std;


// Define a class
class Student {
public:
   // Data member
   string name;


   // Member function
   void display() {
      cout << "Student Name: " << name << endl;
   }
};


int main() {
   Student s1;        // Create an object
   s1.name = "Ravi";   // Assign value to data member
   s1.display();      // Call member function
}
```

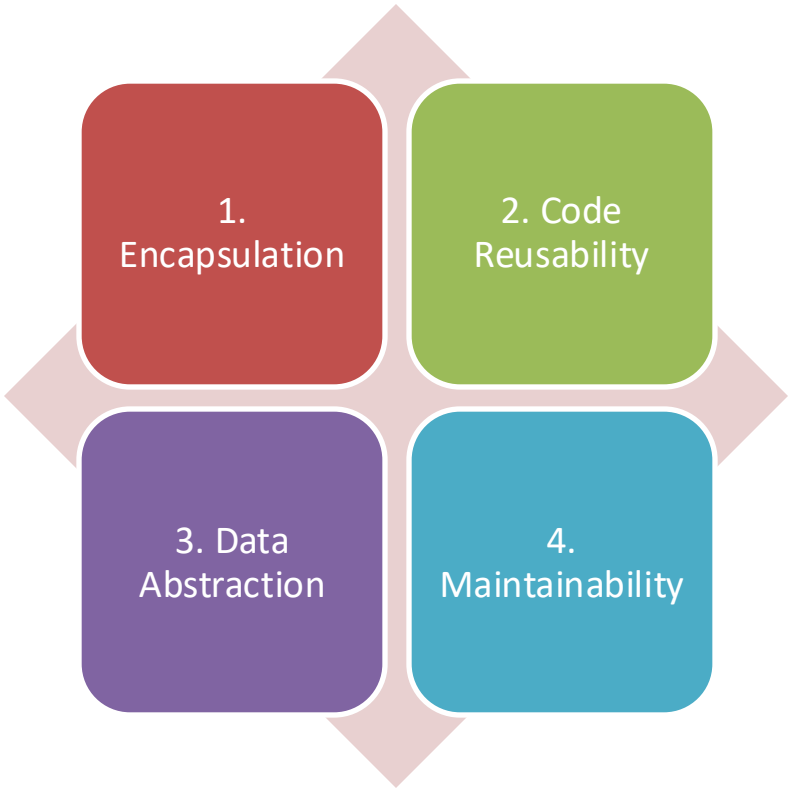
Real-World Scenario: Class as Blueprint

- Just like a blueprint defines the structure of a house, a class defines the structure of an object (e.g., Car class for creating multiple car objects).

Object Lifecycle

- Object lifecycle includes creation, usage, and destruction. C++ manages memory and resources via constructors and destructors.

Advantages of Using Classes and Objects



Encapsulation and Data Management

- Using classes, we can group data and functions logically and hide implementation details from users.

Access Specifiers: private, protected, public

- private: Accessible within the class only
- protected: Accessible within the class and subclasses
- public: Accessible from anywhere

private: Use Case and Example

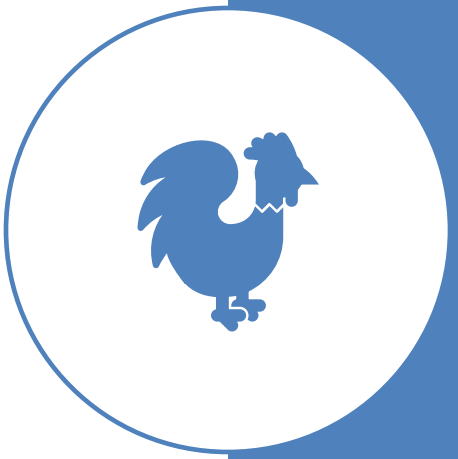
```
class Account {
  private:
    double balance;
};
```

- Ensures data protection.
- Access from inside same class only (practice code example)

protected: Use Case and Example

Used in inheritance :

Accessible in its own class and derived class only .



Inheritance example

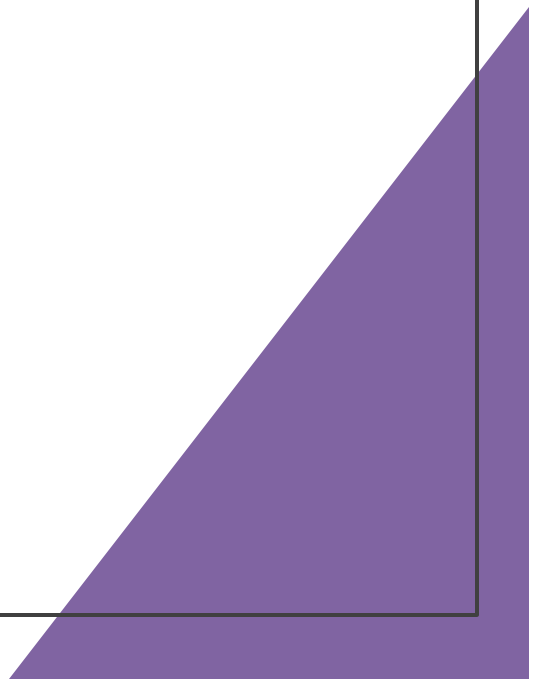
```
#include <iostream>
using namespace std;

class Animal {
protected:
   string name;  // protected: accessible in Dog and Cat, not in main

public:
   void eat() { cout << name << " eats\n"; }

   void setName(string n) { name = n; } // public method to set protected name
};

class Dog : public Animal {
public:
   void bark() {
      cout << name << " barks\n";  // allowed: name is protected
   }
};

class Cat : public Animal {
public:
   void meow() {
      cout << name << " meows\n";  // allowed: name is protected
   }
};
```

```
int main() {
   Dog d;
   d.setName("Bruno");
   d.eat();      // inherited from Animal
   d.bark();     // Dog's own method


   Cat c;
   c.setName("Luna");
   c.eat();      // inherited from Animal
   c.meow();     // Cat's own method


   // cout << d.name; // ❌ Error: 'name' is
      protected, not accessible here
}
```

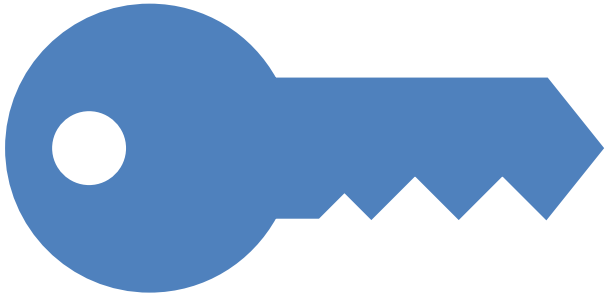
public: Use Case

```
class Person {
  public:
    string name;
};
```

- Allows access from outside the class.



Importance of Access Control

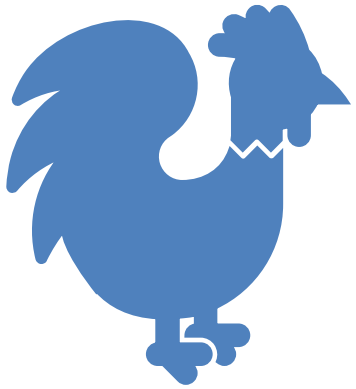


Access control ensures data security and encapsulation by restricting access to class members.

Data Members and Member Functions

Data Members: Variables inside a class

Member Functions: Functions inside a class to operate on data



Data Member and Member function

```
#include <iostream>
using namespace std;

class Employee {
private:
   int id;
   string name;
   float salary;

public:
   // Function to set data members
   void setData(int empId, string empName, float empSalary) {
      id = empId;
      name = empName;
      salary = empSalary;
   }
   // Function to display employee information
   void display() {
      cout << "Employee ID: " << id << endl;
      cout << "Name: " << name << endl;
      cout << "Salary: $" << salary << endl;
   }
};

int main() {
   Employee e1;
   e1.setData(101, "Alice", 50000.0);  // set values
   e1..display();                      // display employee details
}
```

Constructor and Destructor

What is a **Constructor**?

- A **constructor** is a special member function of a class that **runs automatically when an object is created**.
- It is used to **initialize** the object (e.g., set initial values).
- It has the **same name as the class** and **no return type**.

What is a **Destructor**?

- A **destructor** is a special member function that **runs automatically when an object is destroyed** (goes out of scope or deleted).
- It is used to **clean up resources** or perform any final actions before the object is removed.
- It has the **same name as the class**, but preceded by a **tilde ~**, and has **no return type or parameters**.

Constructor and Destructor

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;

    // Constructor: called when object is created
    Person(string n) {
        name = n;
        cout << "Constructor called: " << name << " is
created.\n";
    }

    // Destructor: called when object is destroyed
    ~Person() {
        cout << "Destructor called: " << name << " is
destroyed.\n";
    }

    void display() {
        cout << "Name: " << name << endl;
    }
};
```

```
int main() {
    Person p1("Alice");  // Constructor runs here
    p1.display();
{
     Person p2("Bob"); // Constructor runs here
      p2.display();
}

// Destructor runs here when p2 goes out of scope
    return 0;
}
// Destructor runs here when p1 goes out of scope
```

Real-World Analogy

1. **File Handler**

Constructor: When a program needs to write or read from a file, the constructor automatically opens the file as soon as the file-handler object is created.

Destructor: When the object is no longer needed (goes out of scope), the destructor automatically closes the file to avoid data loss or corruption.

2. **Database Connection**

Constructor: When connecting to a database, the constructor sets up the connection right away as soon as the database object is created.

Destructor: When the program finishes using the database, the destructor closes the connection automatically to free resources and prevent security issues.

What problem it solves

- Constructors and destructors automate the setup and release of resources.

Default Constructor

- A **default constructor** is a constructor that takes **no parameters**. It is **automatically called** when an object is created **without any arguments**.
- If you **don't define any constructor**, C++ provides a default constructor automatically.
- If you define **any other constructor (like parameterized)**, and still want a default one, you must define it manually.

Syntax

```
class ClassName {
public:
   ClassName() {
      // constructor body
   }
};
```

Default Constructor

```
#include <iostream>
using namespace std;

class Student {
public:
    Student() {
        cout << "Default constructor called!" << endl;
    }

    void display() {
        cout << "Student object is ready." << endl;
    }
};

int main() {
    Student s1;     // Default constructor is called automatically
    s1.display();
    return 0;
}
```

Parameterized Constructor

A **parameterized constructor** is a special type of constructor that allows you to **pass values (parameters)** at the time of object creation. It helps in initializing object data members with **user-defined values** instead of default or hardcoded ones.

Syntax

```
class ClassName {
public:
   // Parameterized constructor
   ClassName(type1 arg1, type2 arg2, ...) {
      // initialize members using arguments
   }
};
```

Parametrized Constructor

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int age;

public:
    //  Default constructor (manual)
    Student() {
        name = "Unknown";
        age = 0;
        cout << "Default constructor called\n";
    }

    // Parameterized constructor
    Student(string n, int a) {
        name = n;
        age = a;
        cout << "Parameterized constructor called\n";
    }

    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};
```

```
int main() {
    Student s1;              // Calls default constructor
    s1.display();

    Student s2("Alice", 21);    // Calls parameterized constructor
    s2.display();

    return 0;
}
```

Copy Constructor

A **copy constructor** is a special constructor in C++ that is used to **create a new object as an exact copy of an existing object**.

Syntax:

```
ClassName(const
    ClassName &obj);
```

Scenario: Copying Student Record

- Used in scenarios where deep copy is needed (e.g., duplicating student records).

Copy Constructor

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int age;

    Student(string n, int a) {
        name = n;
        age = a;
    }

    // Copy constructor
    Student(const Student &s) {
        name = s.name;
        age = s.age;
    }

    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};
```

```
int main() {
    Student s1("Ravi", 20);
    Student s2 = s1;  // Copy constructor used

    s1.display();
    s2.display();
}
```

Benefits of Constructor Types

- Provide flexibility in object creation, especially with different initial values or copying.
- **duplicate objects** without retyping data.
- **safely pass and return objects** by value in functions.
- **make backups or temporary versions** of data structures.

Constructor Overloading

- **Constructor overloading** means defining **multiple constructors** in the same class with **different sets of parameters** (number or type).
  It allows objects to be initialized in **different ways**.

Why?

- To create **multiple ways** to initialize an object.
- Makes a class more **flexible and reusable**.

Constructor Overloading

```
#include <iostream>
using namespace std;

class Student {
private:
   string name;
   int age;

public:
   // Default constructor
   Student() {
      name = "Unknown";
      age = 0;
   }

   // Parameterized constructor (1 parameter)
   Student(string n) {
      name = n;
      age = 0;
   }

   // Parameterized constructor (2 parameters)
   Student(string n, int a) {
      name = n;
      age = a;
   }

   void display() {
      cout << "Name: " << name << ", Age: " << age << endl;
   }
};
```

```
int main() {
   Student s1;              // Default constructor
   Student s2("Ravi");      // One-argument constructor
   Student s3("Sita", 20);  // Two-argument constructor


   s1.display();
   s2.display();
   s3.display();
}
```

Advantages of Constructor Overloading

Allows different ways of initializing objects depending on available data.
Flexibility in Object Initialization
You can create objects in different ways (with no data, partial data, or full data).

Improves Code Readability and Usability
Makes the class easier to use in different situations without writing extra methods.

Supports Default and Custom Initialization
You can provide default values using one constructor and custom values using another.

Reduces Code Duplication
No need to write multiple classes or functions for different initializations — just overload the constructor.

Enhances Reusability
A single class can serve multiple needs by offering different initialization options.

Encapsulation of Logic
All initialization logic stays inside the constructors, making your class clean and self-contained.

Array of Objects

An **array of objects** is a collection of multiple objects of the **same class**, stored in **contiguous memory locations** like a regular array.

Why to use

- To manage and process multiple objects easily using a loop.
- Useful for storing a list of students, employees, books, etc.

Example

```
#include <iostream>
using namespace std;

class Student {
public:
   string name;
   int age;

   void setData(string n, int a) {
      name = n;
      age = a;
   }

   void display() {
      cout << "Name: " << name << ", Age: " << age
<< endl;
   }
};
```

```
int main() {
   Student s[3];  // Array of 3 Student objects

   s[0].setData("Ravi", 20);
   s[1].setData("Sita", 22);
   s[2].setData("Ram", 19);

   for (int i = 0; i < 3; i++) {
      s[i].display();
   }

   return 0;
}
```

Static Data Member

- A **static data member** is a variable that is **shared among all objects** of the class.
- There is only **one copy** of the static variable, regardless of how many objects are created.

Features

- Shared by all objects of the class.
- Initialized only once — outside the class.
- It is stored in the **data segment**, not inside each object.
- Can be accessed using the **class name**.

Analogy

- Think of a **classroom** as a class, and each **student** as an object.
  Each student has their own **name** and **roll number** (non-static data).
  But they all share the same **classroom number** (static data) — it's common to all.

Syntax

```
class ClassName {
    static int count;   // Declaration inside class
};

int ClassName::count = 0;   // Definition outside class (required!)
```

Static Member Function

A **static member function** is a function that:

- **Belongs to the class**, not to any object.
- Can be called using the **class name**, without creating an object.
- Can access only **static members** of the class.

Note:

- Static functions **do not have a `this` pointer**.
- Cannot access **non-static variables** or functions.

Static Member Function Syntax

```
class ClassName {
public:
   static void show();   // Declaration
};


void ClassName::show() {   // Definition
   // Code here
}
```

Analogy

Think of a **bank** as a class.
Every account (object) has its own **balance**.
But the bank has a **static function** like `getTotalAccounts()` — which tells how many total accounts exist in the whole bank.
This info doesn't depend on any one account — it belongs to the bank (class).

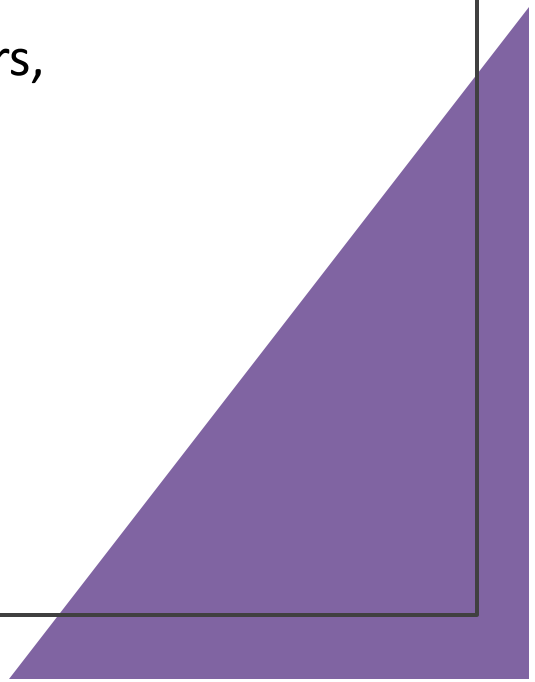
Example

```
#include <iostream>
using namespace std;

class Car {
private:
   string brand;
   static int carCount;   // Static data member

public:
   Car(string b) {
      brand = b;
      carCount++;       // Increases for every object created
   }

   static void showCount() {   // Static function
      cout << "Total Cars Created: " << carCount << endl;
   }
};

// Static variable must be defined outside the class
int Car::carCount = 0;
```

```
int main() {
   Car c1("Toyota");
   Car c2("Honda");
   Car c3("Ford");

   Car::showCount();   // Call without object

   return 0;
}
```

Advantages of Static Members

- Saves memory (only one copy).
- Useful for tracking class-level info (like counters, IDs).
- Easy to access without creating an object.



Encapsulation

- **Encapsulation** is the process of combining **data** and the **functions that operate on that data** into a single unit (i.e., a class) and restricting direct access to the data by making it **private or protected**.
- We use **public methods** (getters and setters) to **control access** to private data.

Benefits:

Data hiding: Keeps internal data safe from unauthorized access.

Code maintainability: Easier to update or change without affecting outside code.

Modularity: Each class handles its own data and logic.

Validation: Allows checks before setting values.

Analogy

Think of a **bank ATM**:

You can **access your money (data)** only via the **ATM interface (functions)**.

You **can't directly open** the cash box (data is hidden).

Example

```
#include<iostream>
using namespace std;

class Student {
private:
   string name;
   int age;

public:
   // Setter functions
   void setName(string n) {
      name = n;
   }

   void setAge(int a) {
      if (a > 0)
         age = a;
   }

   // Getter functions
   string getName() {
      return name;
   }

   int getAge() {
      return age;
   }
};
```

```
int main() {
   Student s;
   s.setName("Aayush");
   s.setAge(20);

   cout << "Name: " << s.getName()
<< endl;
   cout << "Age: " << s.getAge()
<< endl;

   return 0;
```

Getter and Setter

Getter: Returns the value of a private variable.

Setter: Sets or modifies the value of a private variable with validation if needed.

```
class Student {
private:
   int age;

public:
   void setAge(int a) {
      if (a > 0 && a < 150)  // ✅ validation condition
         age = a;
      else
         cout << "Invalid age!" << endl;
   }

   int getAge() {
      return age;
   }
};
```

Friend Function

- A **friend function** is a **non-member function** that has access to the **private and protected members** of a class. It is declared using the `friend` keyword **inside the class** but **defined outside**.
- It is used when:
- An **external function** needs access to private data.
- Two **classes need to share data**.
- You are **overloading operators**.

Friend Function accessing Private Member

```
#include<iostream>
using namespace std;

class Box {
private:
   int width;

public:
   Box() { width = 10; }

   // Declare friend function
   friend void printWidth(Box b);
};

// Define friend function
void printWidth(Box b) {
   cout << "Width: " << b.width << endl;
}

int main() {
   Box b;
   printWidth(b);
   return 0;
}
```

Analogy

A **Patient** has private health data (blood pressure, sugar level, etc.).

A **Doctor** is not part of the Patient class but needs **special access** to read the private data to give treatment.

So, the **Doctor class has a friend function** that is allowed to access the **Patient's private data**.

Advantages of Friend Concept

- Fine-grained access control for debugging, utilities, etc.
- Eg : logger can access private member for logging purpose , which can help in finding errors in code.

What problem friend function solves?

Breaks encapsulation only when necessary, like for testing or inter-class cooperation.

Assignment

Create a class `Car` with private data members `brand` and `year`.
Add public getter and setter functions.

Add a protected data member `engineNumber` with public getter and setter.

Add a static data member `numberOfCars` to track created objects.
Create a static function to return this count.

Write a friend function `compareCars(Car c1, Car c2)` to compare the years of two cars.

Define constructors:

Default constructor

Parameterized constructor

Copy constructor

Create objects using all constructors and demonstrate features.